Intel·ligència Artificial

Windsurf i el paradigma de l'IDE Agèntic

Maria Caldart Deu
Nathaniel Cruz Pascual
Aleix Molins Teulé

TAULA DE CONTINGUTS

1. Descripció del producte i de la innovació **3**
1.1. El Producte 3
1.2. La Innovació 3
2. Descripció general de les tècniques d'IA **3**
2.1. Arquitectura Cascade 3
2.2. Grans Models de Llenguatge (LLMs) 4
2.3. Retrieval-Augmented Generation (RAG) 6
2.4. Sistemes Agèntics i Ús d'Eines (Tool Calling) 8
2.4.1. Protocol MCP 9
3. Com han estat adaptades i utilitzades per a la innovació **10**
4. Per què és innovador i naturalesa de la innovació **11**
5. Impacte de la innovació a l'empresa **12**
5.1. Beneficis 12
5.2. Riscos 13
5.3. Posició en el mercat 14
6. Impacte en l'usuari o societat (Beneficis i Riscos) **16**
6.1. Beneficis per a l'usuari i la societat 16
6.2. Riscos per a l'usuari i la societat 16
7. Anàlisi Crítica: Limitacions Tècniques i Costos **17**
8. Conclusions **18**
9. Bibliografia i Referències **18**

1. Descripció del producte i de la innovació

1.1. El Producte

Windsurf és un Entorn de Desenvolupament Integrat (IDE) desenvolupat inicialment per l'empresa Codeium. A diferència dels editors de codi tradicionals que només afegeixen l'IA com un accessori, Windsurf (agafant com a base l'editor de codi open source Visual Studio Code) ha estat dissenyat amb el propòsit de integrar completament la intel·ligència artificial en la generació i edició de codi.

1.2. La Innovació

La innovació que va introduir aquest editor de codi, que el separa d'altres editors similars com Cursor o Github Copilot, va ser la capacitat de rebre ordres de l'usuari i executar-les en el terminal o efectuant canvis en el codi, sense confirmació humana. És a dir, l'editor va passar a ser capaç d'executar una cadena de tasques de forma seguida, en lloc de tasques demanades una a una amb el usuari que requeririen confirmació. Això va permetre profunditzar molt més la complexitat de les tasques que l'editor de codi podia fer amb una sola instrucció.

La sortida d'aquest IDE va portar la transició cap a un IDE més agèntic, on l'IDE és capaç d'entendre tot el projecte sencer de l'usuari, i és capaç de planificar tasques, obrir i modificar múltiples fitxers simultàniament, i executar comandes directament al terminal sense intervenció humana constant.

2. Descripció general de les tècniques d'IA

2.1. Arquitectura Cascade

L'agent autònom que incorpora aquestes funcions agèntiques dins de la base de Visual Studio Code s'anomena Cascade.

Dins de l'IDE es pot interactuar amb Cascade en dos modes diferents:

- **Mode *code*:** Quan es demana a l'agent que executi una tasca, aquest té completa llibertat per crear, modificar i esborrar arxius, i executar comandes sense haver de demanar confirmació a l'usuari.
- **Mode *chat*:** Està optimitzat per contestar preguntes sobre el teu projecte i sobre principis generals de programació. No té permisos per editar codi o executar comandes, però pot fer suggerències, les quals l'usuari haurà d'insertar manualment.

Aquests dos modes estan suportats per un Gran Model de Llenguatge (LLM) de la teva elecció (Claude, ChatGPT, Gemini,…). Aquests LLMs solen requerir d'una subscripció a la seva API per poder utilitzar-los amb aquests editors agèntics.

Una de les capacitats principals que introdueix Windsurf, en el procés d'integrar aquests LLMs, és la manera en què detecta quines parts del codi són rellevants per passar al model. Pensem que els LLMs tenen finestres de context limitades, per tant, a l'hora d'entendre projectes molt grans estan limitats si el tamany del projecte supera la seva finestra de context.

Per tant, Windsurf millora el tractament del codi a través de l'aplicació de tècniques de RAG (Retrieval-Augmented Generation). Les quals li permeten passar els fragments de codi rellevants als LLMs i eliminar milers de línies innecessàries de codi, que no són rellevants a la tasca.

A continuació farem una explicació més detallada de cada element que composa l'agent Cascade, especificant quin paper juguen en el conjunt d'accions que fa l'agent cada cop que l'usuari envia una ordre.

2.2. Grans Models de Llenguatge (LLMs)

Dins de l'arquitectura Cascade, els LLMs actuen com el cervell de l'agent: és el component que, donat un context complex (fragments de codi rellevants, historial d'accions, estat del terminal i la instrucció de l'usuari), decideix quina és la pròxima acció a executar. Aquesta acció pot ser modificar un fitxer, executar una comanda al terminal, o demanar més informació. Per tant, dins de Cascade, el paper del LLM no és tant el de generador de text, sinó el motor de raonament que coordina tota la cadena de decisions de l'agent.

Windsurf permet escollir entre diversos models comercials per alimentar aquest motor, com ara Claude Opus (Anthropic), GPT-5 (OpenAI), entre d'altres.

La majoria d'aquests models estan basats en l'arquitectura Transformer, la qual va introduir una nova forma de processar seqüències de text substituint les xarxes neuronals recurrents (RNNs) per xarxes neuronals utilitzant mecanismes d'atenció.

En el paper *Attention Is All You Need*, on va ser primer introduïda aquesta arquitectura, s'explica que això permet processar seqüències de text de forma paral·lela i capturar relacions complexes entre paraules molt més eficientment.

La innovació principal del Transformer és la incorporació de la tècnica de self-attention. Aquesta tècnica permet al model analitzar simultàniament totes les paraules d'una seqüència i determinar quines són rellevants entre si, independentment de la distància que les separi. En el context de programació, això és especialment important: el model pot relacionar la declaració d'una variable a la línia 10 amb el seu ús a la línia 2.000, entenent dependències que els models seqüencials anteriors no podien detectar.

De forma general, el transformer incorpora un *Encoder* d'atenció adaptant l'input que reb el LLM i un *decoder* d'atenció sobre l'output generat per la xarxa neuronal. Tot i que alguns models moderns com Claude o GPT utilitzen només el decoder.

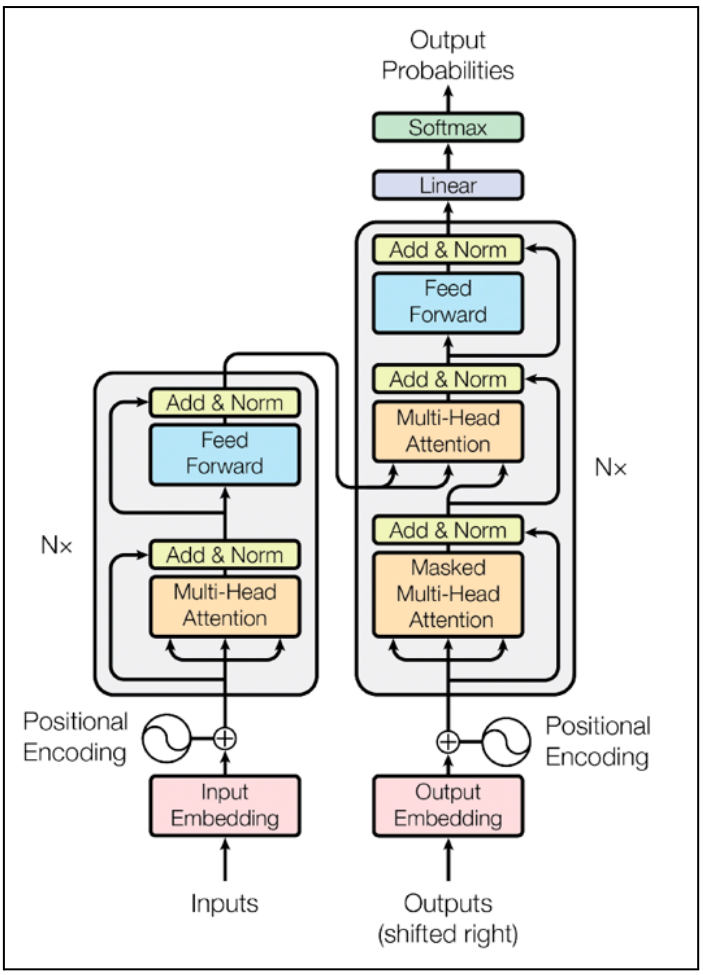


Figura 1: Arquitectura del Transformer plantejada en el paper *Attention Is All You Need*

Tot i això, com hem mencionat breument abans, aquests models presenten una limitació estructural important: la finestra de context, és a dir, el nombre màxim de tokens (fragments de text) que el model pot processar en una sola crida. Fins i tot els models més avançats no poden abarcar projectes grans amb milers de fitxers. Aquesta limitació és precisament la que motiva l'ús de tècniques RAG a Windsurf en complement amb aquesta arquitectura.

Val la pena mencionar també que Windsurf té també la seva pròpia família de models (SWE-1). Aquests models a més incorporen una tècnica diferent en la fase d'entrenament. Enlloc d'entrenar el model únicament per generar un codi que codi un resultat final correcte, se l'entrena també a agafar estats incomplets incorrectes de codi i completar-los. A més, l'objectiu d'aquests models, a més de ser bons en programació i problemes de propòsit general, és de ser-ho en totes les capacitats esperades d'un programador humà: com executar comandes al terminal, i utilitzar eines de test per comprovar el resultat esperat del codi. Per tant, una part gran del seu entrenament també és millorar aquestes capacitats.

Gràcies a això, en completesa de tasques, els models SWE-1 ofereixen capacitats similars a aquests models comercials, sent més eficients que aquests models, i tenint un preu més assequible.

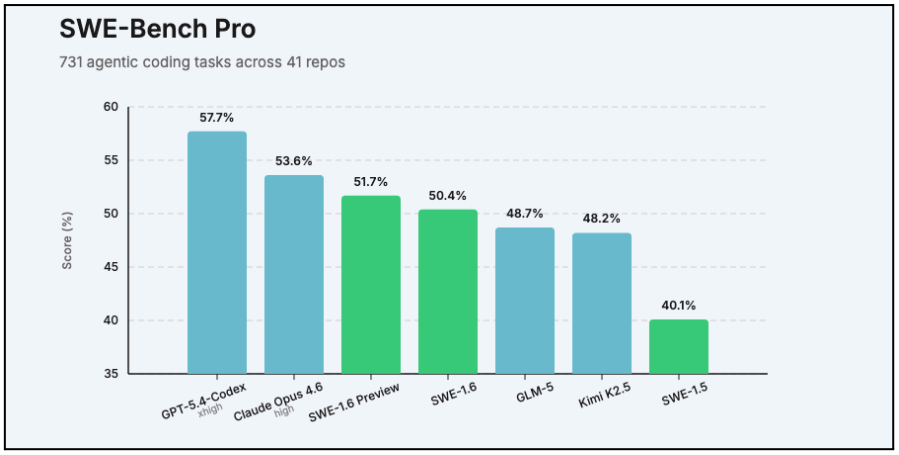


Figura 2: Comparativa entre models comercials i la familia SWE-1, extreta de la pàgina oficial de Windsurf

2.3. Retrieval-Augmented Generation (RAG)

Retrieval-augmented generation, o RAG, es una tècnica que utilitza dades externes objectives per millor la correctesa i rellevància de l'output d'un model. Ho fa seguint la següent seqüència d'accions:

1. **Ingestió** (*Ingestion*): Dades autoratives/objectives es carreguen a una font de dades (típicament en la forma d'un *vector database*). Aquestes dades es divideixen de forma intel·ligent en *chunks* de informació rellevant, permetent accedir fàcilment a mòduls de dades (no el conjunt de dades complet).

2. **Recuperació** (*Retrieval*): Les dades rellevants són recuperades de la font de dades externa basant-se en una ordre de l'usuari.

3. **Ampliació** (*Augmentation*): Les dades recuperades i l'ordre de l'usuari es combinen en un prompt unificat que passar al model. Aquest nou prompt té incorporat el context rellevant a la tasca.

4. **Generació** (*Generation*): El model genera l'output a partir l prompt augmentat, utilitzant el context per donar una resposta més acurada i rellevant.

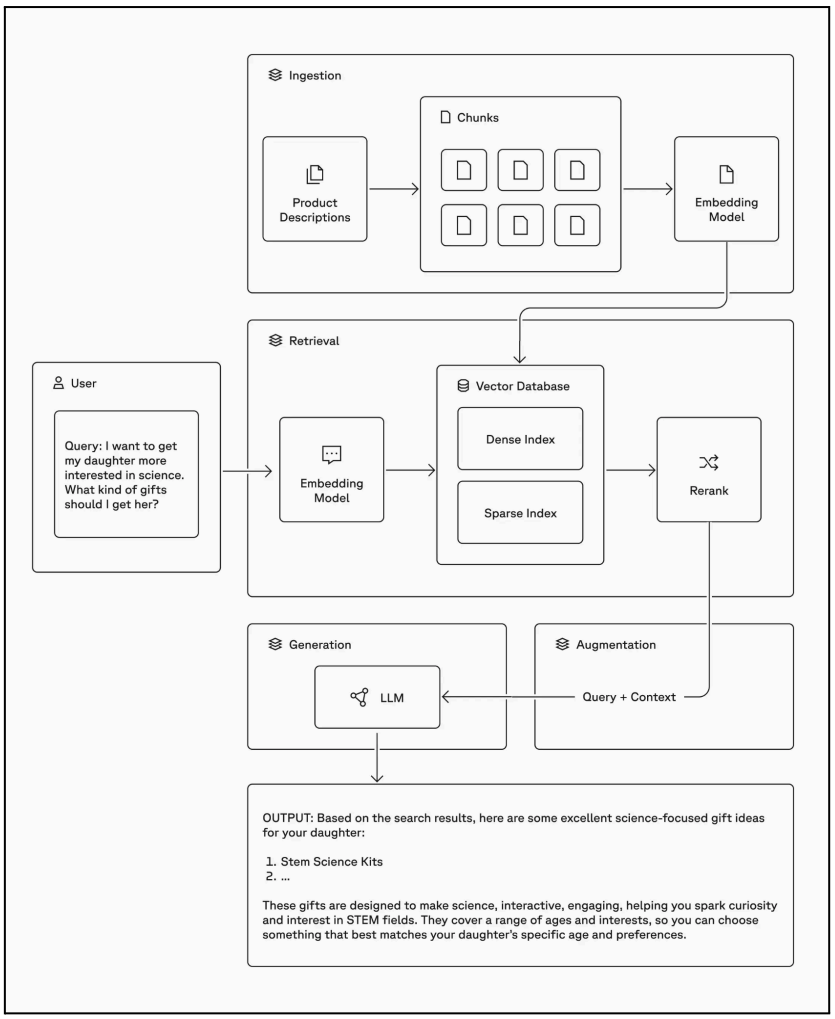


Figura 3: Pipeline que segueix un agent que implementa RAG

La implementació d'aquesta tècnica és molt important dins de Cascade perquè és el mecanisme a través Windsurf agafa el context del projecte que estem programant i passa la part rellevant per la ordre (query) que ha fet l'usuari al LLM.

Aquesta tècnica fa que el contigut que genera el LLM sigui molt més rellevant i acurat al codi. Ja que el model no ha de navegar parts inútils del codi en el prompt i reb només les parts rellevants, augmentant la probabilitat de que es centri en la part correcta del codi i que hi focalitzi les seves capacitats.

La part més important que implementa Windsurf, per tant, és el seu motor de recuperació de dades (*Retrieval*). Ja que la seva capacitat per seleccionar les parts (*chunks*) del codi que realment són rellevants per problema, és el que fa l'agent funcioni bé.

Aquest motor, a més de funcionar de forma autònoma, es pot complementar amb l'ús de comandes integrades dins del sistema, com les mencions (*@-mentions*) que permeten adreçar un arxiu o directori concret al que et vols referir quan fas una *query* a l'agent.

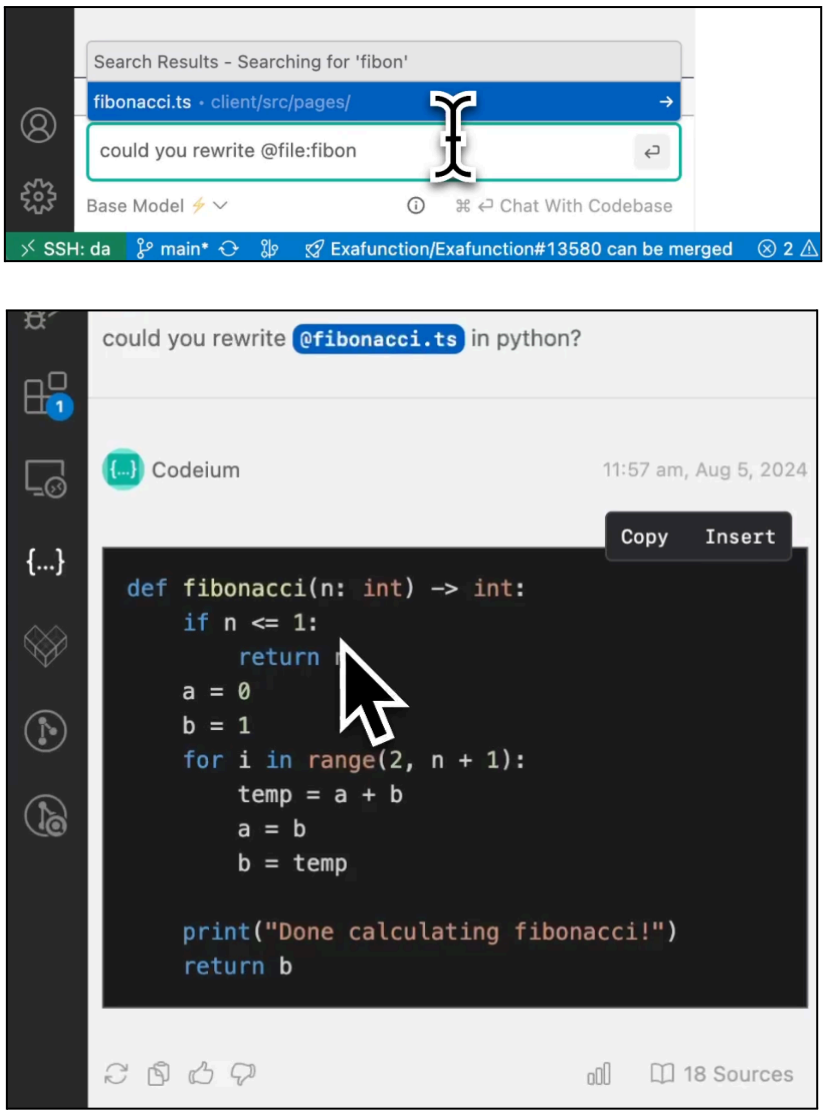


Figura 4: Exemple de l'ús de mencions dins de l'editor de Windsurf, extret de la documentació oficial

2.4. Sistemes Agèntics i Ús d'Eines (Tool Calling)

Fins ara hem vist com el LLM actua com a motor de raonament i com el RAG li proporciona el context rellevant del projecte. Però perquè l'agent pugui actuar de forma autònoma sobre l'entorn, necessita un mecanisme que li permeti anar més enllà de generar text: necessita poder interactuar amb l'ordinador. Això és precisament el que habilita el *tool calling* (ús d'eines).

El *tool calling* és una tècnica que permet als LLMs invocar funcions predefinides durant el procés de generació. En lloc de retornar únicament text pla, el model pot emetre una crida estructurada a una eina concreta (per exemple, "edita el fitxer X", "executa aquesta comanda al terminal", "llegeix el contingut d'aquest directori"), i l'entorn s'encarrega

d'executar-la i retornar el resultat al model. Aquesta capacitat transforma el LLM d'un generador de text en un agent capaç d'actuar.

Dins de Cascade, les eines disponibles inclouen, entre d'altres:

- **Lectura i escriptura de fitxers:** Permet crear, modificar o eliminar arxius del projecte directament.
- **Execució de comandes al terminal:** Permet llançar scripts, instal·lar dependències, executar tests o compilar el projecte.
- **Lectura de errors:** Permet llegir els errors i advertències que retorna el compilador després d'un canvi.
- **Cerca per internet:** L'agent és capaç de buscar documentació o informació relevant a les sol·licituds que se li estiguin fent.
- **Integració amb eines que segueixen el protocol MCP:** L'agent pot comunicar-se i efectuar accions dins de servidors que suportin aquest protocol. Un exemple de servidor MCP popular és Github. De tal manera que, si se li pasen les credencials d'un compte github a l'agent, pot fer commits o recollir informació dels teus repositoris privats.

La clau del comportament agèntic rau en com s'encadenen aquestes eines dins d'un **bucle de raonament autònom**. Quan l'usuari envia una instrucció, el LLM no executa una única acció i s'atura: avalua el resultat de cada eina invocada, i decideix quina és la pròxima acció a partir d'aquest resultat. Aquest cicle de *raonament -> acció -> observació -> raonament* es repeteix fins que l'agent considera que la tasca s'ha completat.

Per exemple, davant la instrucció d'afegir una nova funcionalitat, l'agent escriurà el codi, executarà els tests automàticament, llegirà l'error retornat, corregirà el codi i tornarà a executar els tests, tot sense intervenció de l'usuari.

Aquesta capacitat d'auto-avaluació i correcció és el que diferencia Windsurf dels editors de codi tradicionals amb IA integrada: l'agent tanca el bucle de accions per si sol, sense confirmació humana, fent-se responsable del resultat final.

2.4.1. Protocol MCP

El Model Context Protocol (MCP) és un estàndard open-source per connectar aplicacions que usen LLMs amb sistemes externs. Aquests sistemes poden ser fonts de dades (fitxers locals de la teva màquina o bases de dades externes) o eines (ex: buscadors, calculadores, etc.).

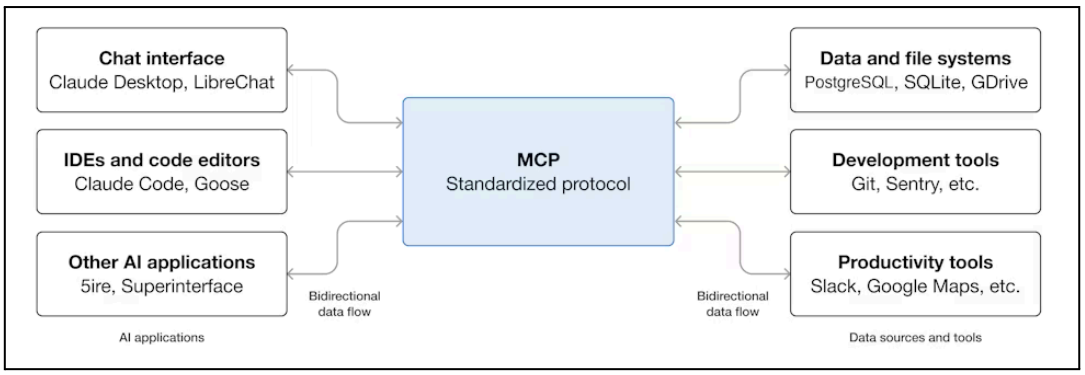


Figura 5: Esquema representatiu del protocol MCP

La integració d'aquest protocol dins de cascade vol dir que aquest es pot comunicar amb qualsevol servidor que l'adopti (Github, PostgreSQL, Slack, Notion, Google Drive, etc.). Això expandeix les capacitats de Cascade com a agent més enllà de les que implementa l'equip de Windsurf, a totes aquelles que suportin aquests servidors.

És a dir, gràcies a la incorporació de MCP, Cascade deixa de ser un agent tancat (limitat a les eines que implementa Windsurf) a un agent obert i extensible per qualsevol servei extern.

3. Com han estat adaptades i utilitzades per a la innovació

Les tècniques descrites al punt anterior no s'han incorporat a Windsurf de forma aïllada, sinó que s'han integrat conjuntament per donar lloc a un sistema coherent on cada component resol una limitació específica dels editors de codi tradicionals.

El primer repte que calia abordar era la **limitació de context dels LLMs**. Projectes de programació reals poden contenir milers de fitxers i centenars de milers de línies de codi, molt per sobre de la finestra de context de qualsevol model. La resposta de Windsurf ha estat aplicar RAG al repositori sencer: en el moment en què l'usuari envia una instrucció, el motor de recuperació indexa el projecte, fragmenta el codi en unitats semàntiques rellevants i selecciona únicament els fragments necessaris per respondre aquella consulta concreta. D'aquesta manera, el LLM rep un context reduït però precís, cosa que augmenta la qualitat de les seves respostes i redueix el risc que el model es perdi en codi irrellevant.

El segon repte era passar de la **generació de text a una acció real** sobre el sistema. Un LLM per si sol només pot produir text. Perquè Cascade pugui crear fitxers, executar comandes al terminal o llegir errors del compilador, cal un mecanisme que tradueixi les decisions del model a operacions concretes. Aquí és on entra el tool calling: Windsurf defineix un conjunt d'eines predefinides (lectura i escriptura de fitxers, execució de comandes, cerca web, etc.) que el model pot invocar de forma estructurada durant el procés de raonament. Cada crida a una eina retorna un resultat que el model processa per decidir el pas següent.

La combinació d'ambdues tècniques és el que composa el comportament autònom de Cascade. El cicle és el següent: el RAG proporciona el context adequat, el LLM raona sobre la tasca i decideix quina eina cal invocar, l'eina s'executa i retorna el resultat, i el LLM torna a raonar a partir d'aquest resultat fins que considera que la tasca s'ha completat. Aquest bucle de raonament, acció, observació no requereix intervenció humana, cosa que permet encadenar tasques complexes a partir d'una sola instrucció.

A més, la incorporació del **protocol MCP** ha permès estendre aquest sistema més enllà de les eines que implementa l'equip de Windsurf. En lloc de construir integracions ad hoc per a cada servei extern (GitHub, Slack, bases de dades, etc.), Cascade pot comunicar-se amb qualsevol servidor que adopti el protocol, convertint l'agent en un sistema obert i extensible.

Finalment, els models propis de Windsurf (**família SWE-1**) representen una adaptació específica en la fase d'entrenament. A diferència dels models comercials generals, SWE-1 s'ha entrenat amb situacions pròpies del flux de treball d'un programador: completat de codi incomplets, execució de comandes al terminal i interpretació dels resultats de les eines de test. Això els permet operar de forma més eficient en el context agèntic de Cascade, amb un cost computacional inferior al dels models comercials equivalents.

4. Per què és innovador i naturalesa de la innovació

La innovació que representa Windsurf no rau en cap de les tècniques que utilitza per separat (els LLMs, el RAG i el tool calling existien prèviament com a tecnologies independents) sinó en la seva integració dins d'un IDE de codi d'una forma diferent als editors de codi anteriors.

Fins a l'aparició de Windsurf, els editors amb IA integrada com GitHub Copilot funcionaven utilitzen la IA passivament: el model suggeria codi i l'usuari decidia si l'acceptava o no. Cada acció requeria confirmació humana. La innovació de Windsurf va ser trencar aquest paradigma i substituir-lo per un de nou: el de l'agent autònom. En aquest nou paradigma, l'usuari no confirma cada pas, sinó que delega una tasca completa i l'agent la du a terme de principi a fi, gestionant els errors, modificant fitxers i executant comandes sense intervenció constant.

Aquesta transició és significativa perquè canvia la naturalesa de la relació entre el programador i l'eina. En el model tradicional, el programador continua sent l'executor de totes les accions; l'IA és un complement. En el model agèntic, el programador passa a ser un supervisor que defineix objectius, mentre l'agent s'encarrega de l'execució. Això permet abordar tasques d'una complexitat molt superior amb una sola instrucció.

Un altre element que reforça el caràcter innovador és l'extensibilitat de l'agent a través de MCP. La majoria de sistemes agèntics anteriors eren sistemes tancats, amb un conjunt fixe

de capacitats definides pels seus programadors. Windsurf, en adoptar un protocol obert, va fer les capacitats del seu agent extensibles a qualsevol servei extern.

Així doncs, classifiquem la naturalesa de la innovació de Windsurf principalment com un ús innovador de tècniques i mètodes ja existents. L'empresa no ha inventat les xarxes neuronals Transformer, ni l'arquitectura RAG, ni les APIs de Tool Calling. La veritable disrupció de la seva naturalesa recau en l'enginyeria d'integració: han agafat aquestes tècniques aïllades i les han orquestrat dins d'un IDE creat des de zero per forjar un paradigma completament nou (l'IDE Agèntic). D'altra banda, amb la creació de la família de models SWE-1, també presenten una innovació menor en el mètode d'entrenament de la IA, ja que adapten l'aprenentatge del model específicament per resoldre estats incomplets de codi i interactuar amb terminals, allunyant-se dels LLMs de propòsit general.

5. Impacte de la innovació a l'empresa

La introducció de Windsurf com a primer IDE veritablement agèntic ha tingut un impacte profund per a la seva empresa creadora, Codeium (i posteriorment per a Cognition AI), transformant radicalment la seva posició de mercat i el seu model de negoci.

5.1. Beneficis

El principal benefici ha estat aconseguir l'avantatge del pioner (*first-mover advantage*) en una nova subcategoria de mercat. Mentre gegants com Microsoft centraven els seus esforços a incrustar GitHub Copilot (un model d'assistència passiva) dins d'entorns heretats, Codeium va captar l'atenció del sector tecnològic demostrant que un IDE construït al voltant de la IA és inherentment superior. Aquest fet va disparar la seva penetració al mercat (assolint l'ús en el 59% de les empreses del Fortune 500) i va generar un enorme atractiu inversor. L'impacte financer directe va culminar a finals de 2025, quan Cognition AI va adquirir Windsurf en una operació estratègica multimilionària per consolidar el seu monopoli en l'enginyeria de programari autònoma. A nivell de negoci, la transició cap a models "Enterprise" de pagament recurrent assegura a l'empresa ingressos constants altíssims gràcies a l'estalvi de temps que prometen als seus clients B2B.





Figura 6 i 7: L'adquisició de Windsurf per part de Cognition AI consolida el lideratge en l'enginyeria autònoma.

5.2. Riscos

Malgrat l'èxit, aquesta innovació comporta riscos estructurals i operatius greus per a l'empresa. El més crític és el cost computacional d'inferència. Processar tot el context d'un repositori mitjançant RAG i enviar desenes de milers de *tokens* a models frontera com Claude Sonnet per a cada tasca autònoma suposa un cost d'infraestructura massiu.

Suposant que una tasca agèntica típica de Cascade consumeix uns 50.000 tokens, i el cost d'un model com Claude Sonnet és de 3$ per milió de *tokens*. El cost operacional d'una acció de Cascade és de 0.15$. Un equip de 10 programadors fent 20 tasques al dia costaria uns $30/dia, que equival a uns $750/mes, considerant 25 dies laborables mensuals. Aquest elevat cost, en comparació amb el cost mensual dels típics models de subscripció (aprox. 20$), exemplifica el risc de l'ús d'una quota mensual com a model de negoci, i la seva possible ineficàcia a llarg termini.

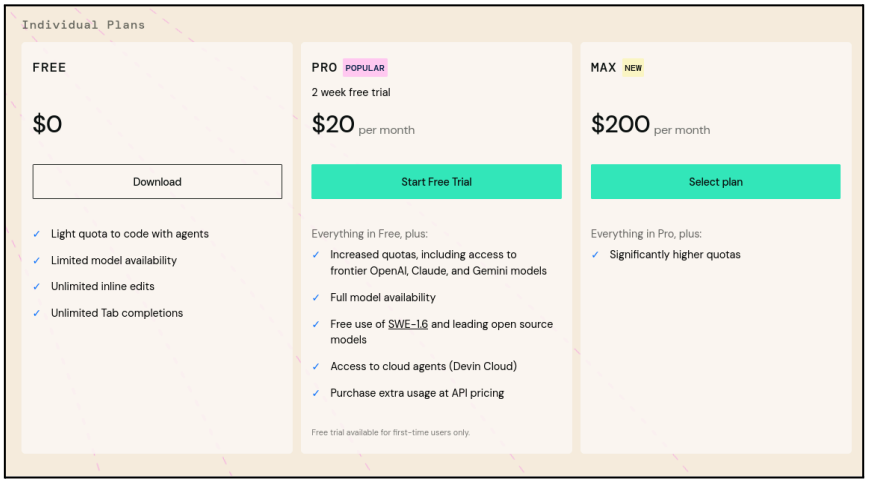


Figura 8: Plans mensuals dels models SWE-1 de Windsurf

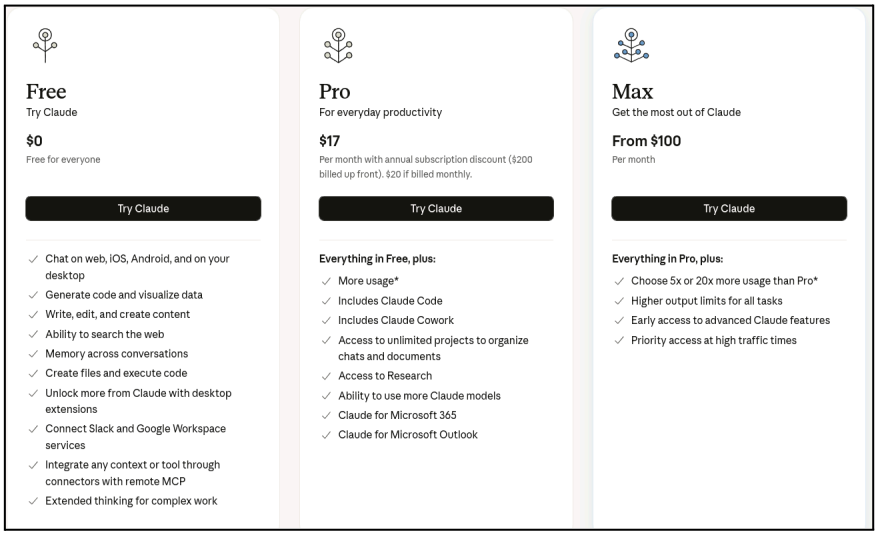


Figura 9: Plans mensuals dels models Claude d'Anthropic

Model	Base Input Tokens	5m Cache Writes	1h Cache Writes	Cache Hits & Refreshes	Output Tokens
Claude Opus 4.7	$5 / MTok	$6.25 / MTok	$10 / MTok	$0.50 / MTok	$25 / MTok
Claude Opus 4.6	$5 / MTok	$6.25 / MTok	$10 / MTok	$0.50 / MTok	$25 / MTok
Claude Opus 4.5	$5 / MTok	$6.25 / MTok	$10 / MTok	$0.50 / MTok	$25 / MTok
Claude Opus 4.1	$15 / MTok	$18.75 / MTok	$30 / MTok	$1.50 / MTok	$75 / MTok
Claude Opus 4 (deprecated)	$15 / MTok	$18.75 / MTok	$30 / MTok	$1.50 / MTok	$75 / MTok
Claude Sonnet 4.6	$3 / MTok	$3.75 / MTok	$6 / MTok	$0.30 / MTok	$15 / MTok
Claude Sonnet 4.5	$3 / MTok	$3.75 / MTok	$6 / MTok	$0.30 / MTok	$15 / MTok
Claude Sonnet 4 (deprecated)	$3 / MTok	$3.75 / MTok	$6 / MTok	$0.30 / MTok	$15 / MTok
Claude Haiku 4.5	$1 / MTok	$1.25 / MTok	$2 / MTok	$0.10 / MTok	$5 / MTok
Claude Haiku 3.5 (retired, except on Bedrock and Vertex AI)	$0.80 / MTok	$1 / MTok	$1.60 / MTok	$0.08 / MTok	$4 / MTok

Figura 10: Cost per milió de tokens de la familia de models de Claude

Un segon risc, que guarda relació amb el primer, és la dependència tecnològica: tot i tenir els models SWE-1, gran part de la capacitat agèntica "Premium" de Windsurf depèn de l'accés a les APIs de tercers (OpenAI o Anthropic), tenint en compte que aquests models són més usats a la plataforma que els seus models SWE-1 propis. Per tant, qualsevol canvi en les polítiques o preus d'aquestes empreses afecta directament el servei.

Finalment, un tercer risc, sovint subestimat, és el risc reputacional derivat dels errors autònoms. En atorgar a Cascade permisos d'escriptura i execució sense confirmació humana constant, qualsevol error de l'agent en un entorn productiu (com l'eliminació accidental de fitxers crítics o la introducció de vulnerabilitats en un commit automatitzat) recau directament sobre la reputació de l'eina i la confiança de l'empresa client. A diferència d'un assistent passiu com GitHub Copilot, on la responsabilitat de cada acció és sempre de l'usuari, el model agèntic de Windsurf assumeix part d'aquesta responsabilitat, la qual cosa representa una exposició legal i comercial nova per a l'empresa.

5.3. Posició en el mercat

Windsurf va irrompre com un agent disruptor. Va obligat als monopolis establerts a reaccionar (com l'evolució forçada de GitHub Copilot cap a models de xat més interactius) i actualment lidera l'avantguarda de la programació juntament amb el seu rival més directe, Cursor.

Aquests tres IDEs (Windsurf, Cursor i Visual Studio+Copilot) destaquen cadascun en àmbits de mercat diferenciats. Més endavant veurem en la taula de la figura 11 les característiques que els diferencien.

Cursor té un 31% de quota de mercat i ha assolit més de 2.000 milions d'ingressos anualitzats al febrer de 2026. Windsurf, d'altra banda, compta amb més d'1 milió de desenvolupadors i 4.000 clients Enterprise (s'enfoca més en l'entorn empresarial que en una amplia base d'usuaris).

Copilot, per la seva banda, manté un 42% de quota de mercat, gràcies al seu preu molt més accessible (10$ al mes vs 20$/mes de Cursor i Windsurf) i a la seva visibilitat (integració directa a Visual Studio Code).

FEATURE	VS CODE + COPILOT	CURSOR	WINDSURF
Autocomplete Technology	Ghost Text using multiple AI models	Supermaven-powered predictive edits	Supercomplete (SWE-1-mini)
Autocomplete Latency	~300ms	<200ms (45ms p99)	<150ms
Chat Interface	Copilot Chat (multi-model)	Integrated codebase-aware chat	Conversational Cascade interface
Multi-File Editing	Limited (Copilot Edits)	Excellent (Composer)	Good (Cascade Agent)
Context Window	64K tokens	200K - 1M tokens	200K - 1M tokens
Starting Price (Pro)	$10.00/month	$20.00/month	$20.00/month (as of March 2026)
Free Tier	2,000 completions + 50 premium requests/month	Constrained agent access	25 credits/month (~100 prompts)
IDE Support	VS Code, JetBrains, Neovim, CLI	Based solely on a VS Code fork	VS Code fork, JetBrains
Large Codebase Handling	Best for 100K+ file monorepos	Best for deep refactoring	Best for following patterns

Figura 11: Comparativa entre els IDEs Copilot, Cursor i Windsurf

En quant a les capacitats tècniques de cada IDE, Cursor i Windsurf es diferencien de Copilot per tenir una finestra de context de fins a 1M de tokens, enfront dels 64K de Copilot, cosa que els permet operar sobre bases de codi molt més grans sense perdre coherència.

I en termes de qualitat agèntica, cada eina excel·leix en un àmbit diferent: Cursor és superior en la refactorització i construcció de codi des de zero (71% de tassa de compleció de tasques), mentre que Windsurf destaca en la detecció de patrons i la compleció de codi ja existent (68% de tassa de compleció de tasques).

Aquesta especialització de Windsurf es correspon amb l'enfoc que hem vist que donen en l'entrenament dels models SWE-1 (sobre estats incomplets de codi). Gràcies a això, Windsurf està situada en una posició competitiva diferenciada pels equips que treballen amb bases de codi existents, mentre que Cursor resulta més adequat per a projectes nous que requereixen refactoritzacions profundes.

Finalment, cal destacar que en març de 2026 els preus de Windsurf i Cursor (rivals directes) van convergir, degut a que Windsurf va decidir pujar el seu pla Pro de 15$ a 20$/mes, igualant-se amb Cursor. Aquest canvi és indicador que el focus de les empreses d'aquest sector està començant a ser la rentabilització del producte, enlloc d'oferir preus competitius amb l'objectiu d'ampliar la seva base d'usuaris. El que es relaciona amb la preocupació que hem mencionat anteriorment de que no es puguin rentabilitzar els costos de les crides d'API amb plans mensuals.

6. Impacte en l'usuari o societat (Beneficis i Riscos)

La integració d'agents com Cascade no només altera les eines de treball, sinó que redefineix l'ofici de l'enginyeria de programari a escala global, afectant tant els professionals (usuaris directes) com l'ecosistema tecnològic de la societat.

6.1. Beneficis per a l'usuari i la societat

- **Democratització de la creació tecnològica:** L'impacte més revolucionari és que es rebaixa dràsticament la barrera d'entrada per desenvolupar solucions de programari. Persones amb coneixements tècnics limitats poden materialitzar idees complexes, ja que Windsurf s'encarrega de la sintaxi i l'arquitectura de base. Això fomenta la creació de petites *start-ups* i la ràpida digitalització d'idees a la societat.

- **Canvi de paradigma (D'obrer a arquitecte):** Multiplica de manera exponencial la productivitat de l'enginyer. L'agent Cascade mitiga totalment l'esforç repetitiu i burocràtic (*boilerplate code*, configuració d'entorns, migracions tedioses). Això allibera l'usuari perquè inverteixi el 100% de la seva capacitat cognitiva en el disseny de l'arquitectura, la lògica de negoci i la creativitat tecnològica, aportant molt més valor a la societat en menys temps.

- **Resolució proactiva d'errors:** Disminueix la frustració de l'usuari, ja que l'agent, al poder llegir el terminal autònomament, corregeix errors de compilació i dependències que històricament podien bloquejar un desenvolupador durant hores.

6.2. Riscos per a l'usuari i la societat

- **L'Erosió del coneixement (Efecte "Caixa Negra" i Sobre-dependència):** Aquest és l'impacte sociològic més perillós dins del sector. Si les noves generacions de programadors júniors deleguen sistemàticament la implementació en Cascade, no desenvoluparan una comprensió profunda de les bases fonamentals de la informàtica, de la memòria o dels algoritmes de baix nivell. A llarg termini, es pot crear una societat de desenvolupadors incapaços de depurar o entendre codi heretat complex quan la IA falli o pateixi al·lucinacions.

- **Riscos crítics de ciberseguretat:** En atorgar a un agent d'IA permisos d'escriptura globals i capacitat per executar comandes al terminal local, s'obren escletxes de seguretat sense precedents. L'usuari s'exposa al risc que la IA pateixi injeccions de *prompts* (*prompt injection*) a través de codi extern manipulat, portant a l'execució d'ordres malicioses a l'ordinador de l'usuari. A més, hi ha el perill constant de filtració de dades sensibles i claus d'autenticació privades cap als servidors del LLM en el procés de lectura de context (RAG).

- **Impacte socio-laboral:** A nivell macroeconòmic, la necessitat de contractar perfils "Picadors de codi" (programadors inicials la tasca dels quals era fer tasques repetitives) es redueix dràsticament. Això genera incertesa laboral per als estudiants

i treballadors en posicions d'entrada del sector informàtic, forçant-los a reciclar-se cap a rols d'enginyeria de sistemes o direcció de projectes abans d'hora.

7. Anàlisi Crítica: Limitacions Tècniques i Costos

A més dels riscos de seguretat i de la possible erosió del coneixement en els programadors novells esmentats anteriorment, la implementació d'IDEs agèntics a gran escala s'enfronta a barreres tècniques i econòmiques significatives que la indústria encara ha de resoldre:

- **Cost Computacional i Econòmic:** El sistema de "Fast Context" requereix processar milers de línies de codi i dependències cada vegada que l'agent pren una decisió. Utilitzar models frontera (com Claude Sonnet) per llegir el context complet del projecte de forma recurrent comporta un cost econòmic elevadíssim en concepte de crides a l'API i consum de tokens. Aquest cost es dispara a causa de l'escalat quadràtic ($O(N^2)$) propi d'aquests models: si es duplica la quantitat de text d'entrada, els recursos de memòria i computació necessaris es quadrupliquen. Com s'observa a la *Figura 12*, això provoca una saturació del mecanisme d'atenció (*Attention Mechanism*), ja que cada nou *token* s'ha de relacionar de forma creuada amb tots els *tokens* anteriors de la finestra. Aquest procés satura ràpidament la memòria RAM de les targetes gràfiques (VRAM) mitjançant el buidat continu de la memòria d'execució (KV Cache). En conseqüència, les solucions corporatives basades exclusivament en aquest paradigma poden tenir costos d'operació prohibitius si no s'optimitza el volum de context enviat.

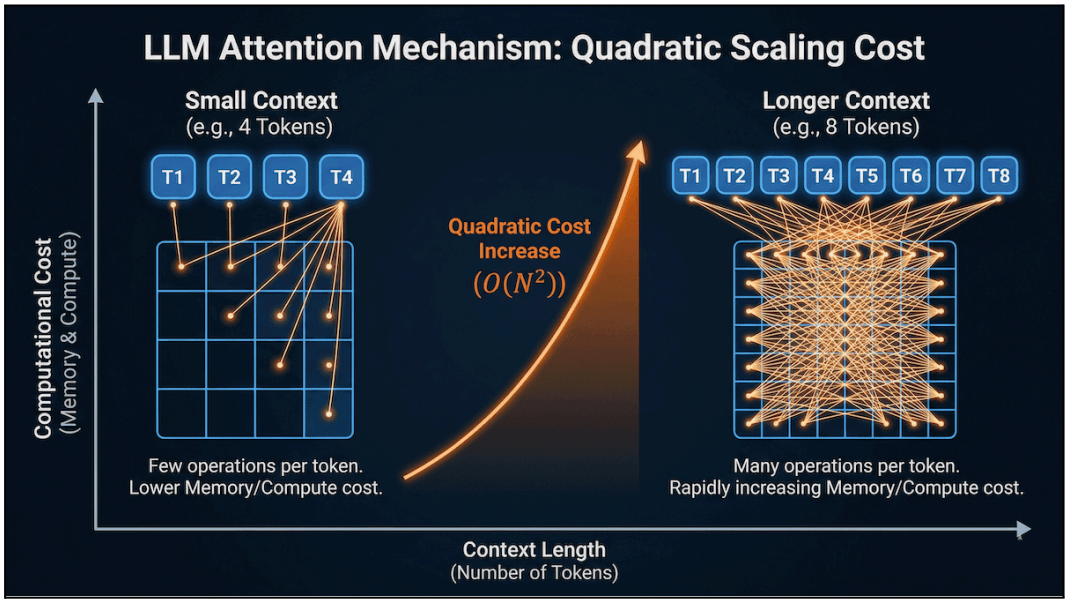


Figura 12: Saturació del mecanisme d'atenció i increment del cost quadràtic segons la longitud del context.

- **Desincronització en Arquitectures Monolítiques (Codi *Legacy*):** Tot i que Windsurf excel·leix en projectes moderns i modulars (com ara entorns web actuals

basats en components i microserveis), l'agent mostra limitacions severes quan ha d'interactuar amb codi heretat (legacy code) amb arquitectures monolítiques antigues. En aquests entorns, on les dependències no estan clarament declarades i el codi "espagueti" és freqüent, l'indexat semàntic RAG perd efectivitat. Això pot provocar "al·lucinacions" on l'IA refactoritza una part del codi trencant silenciosament funcionalitats en altres parts del sistema.

8. Conclusions

L'anàlisi detallada de Windsurf ens permet concloure que no ens trobem davant d'una simple millora incremental en l'autocompletat de codi, sinó davant d'una innovació disruptiva que canvia la pròpia naturalesa de com interactuem amb el programari. La integració d'arquitectures agèntiques profundament arrelades en el mateix nucli de l'IDE, permetent la planificació, execució i validació autònoma de tasques complexes, redueix dràsticament la fricció del procés de desenvolupament.

Aquesta tecnologia assenta les bases del que serà ineludiblement l'estàndard del sector a curt termini. Gràcies a sistemes com Cascade, el rol de l'enginyer informàtic evolucionarà: la tasca de "picar codi" manualment i resoldre errors de sintaxi serà delegada a la màquina, mentre que el professional humà s'elevarà cap a tasques d'alt nivell, com la supervisió, el disseny d'arquitectures escalables i la resolució de problemes de negoci purament lògics i creatius. No obstant això, per assolir l'adopció universal a nivell corporatiu, la indústria haurà de trobar un punt d'equilibri entre l'espectacular productivitat obtinguda, l'estricta gestió de la ciberseguretat i l'elevat cost de la computació d'inferència contínua.

9. Bibliografia i Referències

[1] Windsurf. (s.d.). *Cascade*. Windsurf Docs. Recuperat de: https://docs.windsurf.com/windsurf/cascade/cascade

[2] Windsurf. (2025). *SWE-1: Our first frontier models*. Recuperat de: https://windsurf.com/blog/windsurf-wave-9-swe-1

[3] Windsurf. (2025). *Introducing SWE-1.5: Our fast agent model*. Recuperat de: https://windsurf.com/blog/swe-1-5

[4] Windsurf. (2025). *AI models in Windsurf Cascade*. Windsurf Docs. Recuperat de: https://docs.windsurf.com/windsurf/models

[5] Windsurf. (s.d.). *Windsurf context awareness overview*. Windsurf Docs. Recuperat de: https://docs.windsurf.com/context-awareness/windsurf-overview

[6] Windsurf. (s.d.). *MCP in Cascade*. Windsurf Docs. Recuperat de: https://docs.windsurf.com/windsurf/cascade/mcp

[7] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention is all you need*. arXiv. https://doi.org/10.48550/arXiv.1706.03762

[8] Choudhury, R., Baronio, C., Pan, B., Lee, S., Lu, E., Cao, S., Li, J., Wang, A., Zweiger, A., Wang, R., Chang, G., & Alberti, S. (2026, abril 7). *Introducing SWE 1.6: Improving model UX*. Cognition AI. https://cognition.ai/blog/swe-1-6

[9] Lewis, P., et al. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. arXiv. https://doi.org/10.48550/arXiv.2005.11401

[10] Pinecone. (s.d.). *Retrieval augmented generation*. Recuperat de: https://www.pinecone.io/learn/retrieval-augmented-generation/

[11] Stryker, C. (s.d.). *What is tool calling?* IBM Think. Recuperat de: https://www.ibm.com/think/topics/tool-calling

[12] Model Context Protocol. (s.d.). *What is the Model Context Protocol (MCP)?* Recuperat de: https://modelcontextprotocol.io/docs/getting-started/intro

[13] TechCrunch. (2025). Cognition, maker of the AI coding agent Devin, acquires Windsurf. Recuperat de: https://techcrunch.com/2025/07/14/cognition-maker-of-the-ai-coding-agent-devin-acquires-windsurf/

[14] Dao, T., et al. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. arXiv. Recuperat de: https://arxiv.org/abs/2205.14135

[15] Anthropic. (2026). *Plans & Pricing*. Claude. Recuperat de: https://claude.com/pricing

[16] Anthropic. (2026). *Pricing - Claude API Docs*. Anthropic Platform Documentation. Recuperat de: https://platform.claude.com/docs/en/about-claude/pricing

[17] Cognition AI. (2026). *Plans and Pricing*. Windsurf. Recuperat de: https://windsurf.com/pricing

[18] Carter, A. (8 d'abril de 2026). *Cursor vs VS Code vs Windsurf: Choosing your AI code editor in 2026*. daily.dev. Recuperat de: https://daily.dev/blog/cursor-vs-vs-code-vs-windsurf-ai-code-editor-comparison/